

Fiche de démonstration

Déroulé de la séance avec l'encadrant

SINGHE PENKA Hendrix Donavan

Dépôt GitHub : [HendrixPenka2/credix](#)

Table des matières

Fiche de démonstration	1
1. Objectifs et déroulé de la séance	1
2. Avant la séance : les préparatifs indispensables	1
3. Partie 1 : vérifier que le projet fonctionne depuis Git	2
4. Partie 2 : comprendre comment le projet a été conçu	3
5. Partie 3 : tester la méthode sur le MVP	4
6. Questions probables et éléments de réponse	5
7. Liste de contrôle de la séance	5

Fiche de démonstration

Déroulé de la séance avec l'encadrant : vérifier le projet, comprendre la méthode, tester la méthode sur un exemple.

1. Objectifs et déroulé de la séance

L'encadrant veut vérifier **trois choses**, dans cet ordre :

Partie	Question de l'encadrant	Ce qu'on lui montre	Document
1	« <i>Ce qui a été fait est-il bien sur Git, fonctionne-t-il et est-il utilisable ?</i> »	Le projet récupéré depuis GitHub, lancé et testé	README
2	« <i>Comment avez-vous procédé pour y arriver ?</i> »	La méthode de travail avec Claude, avec les vrais documents du projet	Guide 2 (et guide 1 pour les outils)
3	« <i>Cette méthode marche-t-elle vraiment ? Montrez-le-moi.</i> »	Une conception réduite (MVP) confiée à Claude Code, en direct	Guide 3 (MVP)

Répartition indicative du temps (à ajuster) : partie 1, environ 15 minutes ; partie 2, environ 25 minutes ; partie 3, environ 40 minutes.

2. Avant la séance : les préparatifs indispensables

Le point critique : la première installation est longue. La construction des conteneurs Docker télécharge environ 700 Mo et a duré **environ 50 minutes** sur une connexion lente lors de nos essais. Elle ne doit **jamais** être lancée en direct pendant la séance. **Elle doit être faite avant.**

2.1 Accès au dépôt (à faire la veille)

1. **Inviter l'encadrant** sur GitHub : ouvrir le dépôt, puis *Settings, Collaborators, Add people*, et saisir son nom d'utilisateur GitHub. Le dépôt est **privé** : sans invitation acceptée, il ne peut pas le cloner.
2. Lui demander de **créer un jeton d'accès personnel** (README, section 4.1) : sans lui, Git refuse le mot de passe du compte.
3. Se rappeler qu'un collaborateur d'un dépôt personnel peut **lire et modifier** le dépôt.

2.2 Ordinateur de l'encadrant (idéalement la veille)

À vérifier	Minimum
Système	Linux 64 bits
Espace disque libre	10 Go (la construction a besoin d'environ 4 Go de marge temporaire à la fin)
Outils	Git, Docker et Docker Compose, Node.js 18.18 ou plus, curl, OpenSSL (README, section 3)

À vérifier	Minimum
Ports libres	3001, 8080, 8001, 5000, 27017

Lancer, la veille, la première installation en suivant le README (sections 4 à 6). Le README indique, à chaque étape, ce qu'il faut voir. Le lancement suivant ne prend qu'une minute environ.

2.3 Pour la partie 3 (Claude Code)

- **VS Code 1.94 ou plus** avec l'extension **Claude Code** installée (guide 1, section 7) ;
- un **abonnement Claude payant** (Pro au minimum) ou un compte Console : sans lui, Claude Code ne démarre pas ;
- **être connecté** dans l'extension avant la séance ;
- avoir **copié** docs/guides/03_conception_MVP.md dans un dossier de travail vide, sous le nom docs/conception_mvp.md.

2.4 Côté auteur

- Garder **votre propre pile en marche** (Docker et frontend) : elle sert de **plan B** si l'installation de l'encadrant n'est pas terminée.
- Avoir sous la main : le lien du dépôt, le README en PDF, les guides en PDF, le journal et les documents de passation du projet, l'abonnement Claude.
- Une connexion Internet stable.

2.5 Plan B

Problème	Solution
L'installation de l'encadrant n'est pas terminée	Faire la partie 1 sur votre machine (partage d'écran), et laisser l'encadrant terminer son installation pendant les parties 2 et 3
Pas d'abonnement Claude disponible pour l'encadrant	Faire la partie 3 sur votre compte (partage d'écran), l'encadrant relit et dirige
Claude Code est lent ou saturé (limite d'usage atteinte)	Vérifier votre consommation (guide 1, section 4.3) ; se limiter au plan et au module M2

3. Partie 1 : vérifier que le projet fonctionne depuis Git

Objectif : l'encadrant constate que ce qui est sur GitHub se lance et se teste.

3.1 Déroulé

Étape	Action	Référence
1	Cloner le projet	README, section 4.2
2	Vérifier la copie : 16 fichiers de modèle	README, section 4.3
3	Vérifier l'état du backend : <code>docker compose ps</code> , les logs, <code>/health</code>	README, section 5.3
4	Premier lancement (administrateur et données de démonstration)	README, section 5.4
5	Vérifier le frontend	README, section 6.4

Étape	Action	Référence
6	Parcours de test : connexion, création d'un agent, scoring de HC-100001, dossier en revue avec HC-100241, journal d'audit	README, section 7

3.2 Ce que l'encadrant doit constater

Contrôle	Réussi si...
Récupération Backend	Le <code>git clone</code> réussit ; 'ls scoring-backend/artefacts 4 conteneurs Up, l'API healthy ; /health : "statut": "ok", 27 variables, 27 phrases, "mock_mode": false
Frontend	La page de connexion s'affiche ; les trois espaces (Agent, Superviseur, Administrateur) sont accessibles
Scoring	HC-100001 : score 603, ACCORDÉ ; HC-100241 : score 571, EN REVUE ; HC-101327 : score 536, REFUSÉ
Traçabilité	Le journal d'audit montre connexions, création de compte, scorings et décision du superviseur

3.3 Si quelque chose ne marche pas

Ne pas improviser : ouvrir la **section 10 du README** (dépannage), qui liste les symptômes et les solutions vérifiées. Les cas les plus probables sont un port déjà utilisé, un manque d'espace disque, et l'oubli de redémarrer l'API après le chargement des données.

4. Partie 2 : comprendre comment le projet a été conçu

Objectif : l'encadrant comprend la **méthode**, pas seulement le résultat. On s'appuie sur le **guide 2**.

4.1 Les cinq messages à faire passer

N°	Message	Preuve à montrer
1	On conçoit avant de coder , et on écrit tout en Markdown	Le cahier des charges et la conception ; le README lui-même
2	Claude propose, l'humain décide : aucun livrable sans accord explicite	Les instructions du projet (section 5, workflow de collaboration)
3	Le projet tient sur des sessions , grâce à des documents : instructions, journal, passations, registre des décisions	Le prompt de démarrage de session, le rapport de passation, le journal
4	Chaque choix est justifié et classé par niveau de preuve	L'exemple du WoE (guide 2, section 8)
5	Claude se trompe ; la méthode permet de le voir	Le tableau des erreurs réelles (guide 2, section 14)

4.2 Les fichiers à ouvrir pendant la présentation

- les **instructions du projet** Claude (six sections) ;
- un **prompt de démarrage de session** et un **rapport de passation** ;
- le **journal de bord** et le **journal de rédaction** (règles actées, pièges) ;
- le **registre des décisions méthodologiques** ;
- le fichier `scanner.sh` et l'instantané de contexte qu'il produit.

4.3 La démonstration des outils

Montrer, dans l'ordre : le **Projet** claude.ai (instructions, base de connaissances) ; la page **Utilisation** (limites de session) ; **Claude Code** dans VS Code et son **mode Plan** (guide 1).

5. Partie 3 : tester la méthode sur le MVP

Objectif : montrer que la méthode **fonctionne sur un cas qu'on ne connaît pas**. L'encadrant regarde, ou fait lui-même, ce qui suit. On s'appuie sur le **guide 3**.

5.1 Protocole pas à pas

Étape	Action	Durée indicative
1	Créer un dossier vide <code>demo_mvp</code> , l'ouvrir dans VS Code	1 min
2	Y copier <code>03_conception_MVP.md</code> sous <code>docs/conception_mvp.md</code>	1 min
3	Ouvrir Claude Code, passer en mode Plan (indicateur de mode)	1 min
4	Donner le prompt de l'annexe B du guide 3	1 min
5	Lire le plan proposé et le juger avec la grille ci-dessous	5 à 10 min
6	Corriger si besoin (commentaires dans le plan), puis approuver	5 min
7	Demander la réalisation du module M2 (données et modèle)	5 à 10 min
8	Vérifier le résultat : l'AUC de test et la reproductibilité	3 min

5.2 Grille d'évaluation du plan (étape 5)

Question	Oui / Non
Le plan couvre-t-il les six modules (socle, données et modèle, service de scoring, sécurité, API, interface) ?	
Les modules sont-ils ordonnés selon leurs dépendances (M1, puis M2, puis M3...) ?	
Chaque module a-t-il ses fichiers , ses tests et son critère de fin ?	
Le plan reprend-il les exigences (EF et ENF) du document, sans en inventer ?	
Signale-t-il des ambiguïtés du document ? (un bon signe)	
Ne crée-t-il aucun fichier à ce stade (mode Plan) ?	

5.3 Critères de succès du module M2

Vérification	Attendu
Le script de génération produit 3 000 clients	Fichier <code>clients_synthetiques.csv</code> de 3 000 lignes
L'AUC de test affichée	Au moins 0,70 (environ 0,75 avec les données de l'annexe A)
Deux exécutions successives	Mêmes coefficients à la troisième décimale
Les tests du module	Ils passent

5.4 Si le plan ou le module n'est pas satisfaisant

C'est **normal et instructif** : c'est ce que la méthode est censée révéler.

1. **Identifier ce qui manque** dans le plan (un module oublié, un ordre incohérent).
2. **Le dire à Claude** en mode Plan (« il manque ... », « M3 dépend de M2 »), ou par commentaire dans le plan.
3. Si le document lui-même est ambigu, **le corriger** : la conception est la source de vérité.
4. **Recommencer le plan**, puis approuver.

5.5 Ce que la séance ne cherche pas à prouver

Que Claude écrit un système parfait en une passe. Elle cherche à prouver que **la méthode (conception écrite, mode Plan, un module à la fois, tests)** donne un travail **vérifiable et repreneable**.

6. Questions probables et éléments de réponse

Question de l'encadrant	Éléments de réponse
<i>Pourquoi concevoir avant de coder ?</i>	Une conception écrite est relisible et corrigeable à coût presque nul ; du code écrit sans conception cache ses erreurs. La conception est aussi la source de vérité que Claude Code lit.
<i>Comment savez-vous que Claude ne s'est pas trompé ?</i>	On ne le croit pas sur parole : chiffres vérifiés contre l'exécution réelle, tests, relecture, registre des décisions. Des erreurs réelles ont été trouvées ainsi (guide 2, section 14).
<i>Que vaut la littérature apportée par Claude ?</i>	Elle est un point de départ . Chaque référence importante est vérifiée, et les incertaines restent marquées « à vérifier ».
<i>Peut-on refaire la démarche sur un autre sujet ?</i>	Oui : le guide 2 donne les prompts, les modèles de documents et le cycle de session. Le MVP en est un exemple complet.
<i>Combien cela coûte-t-il ?</i>	Un abonnement Pro à 20 \$ par mois suffit pour commencer ; Max (100 ou 200 \$) si l'usage de Claude Code est intensif (guide 1, section 3). Prix vérifiés le 19 septembre 2026.
<i>Et la confidentialité des données ?</i>	CREDIX a été conçu sur un jeu de données public (Home Credit), sans données de clients réels. Ne jamais confier de données réelles ni de secrets à un outil externe sans autorisation.
<i>Que fait CREDIX que le MVP ne fait pas ?</i>	Deux flux (score et détection des profils atypiques), WoE, calibration, indice de couverture ρ_c , trois rôles, base documentaire, PDF (guide 3, section 13).
<i>Peut-on modifier le projet ?</i>	Oui : README, section 9 (Git), et méthode du guide 2, section 13 (toujours en mode Plan).

7. Liste de contrôle de la séance

La veille

- ☐ L'encadrant est **invité** sur le dépôt et a accepté l'invitation
- ☐ Il a créé son **jeton d'accès**
- ☐ Sa **première installation** est lancée, ou terminée (`docker compose ps` : 4 conteneurs, API healthy)
- ☐ VS Code, l'extension Claude Code et l'abonnement sont prêts
- ☐ Votre propre pile fonctionne (plan B)
- ☐ Espace disque suffisant (au moins 10 Go libres) des deux côtés

Pendant la séance

- ☐ Partie 1 : `git clone`, `/health`, connexion, un scoring
- ☐ Partie 2 : les cinq messages, les vrais documents du projet
- ☐ Partie 3 : plan en mode Plan, grille d'évaluation, module M2

Après la séance

- ☐ Noter les remarques et corrections de l'encadrant
- ☐ Mettre à jour les documents concernés (le `.md` est la source ; le `.tex` et le PDF se régénèrent avec `docs/build/build_all.sh`)

- ☐ Enregistrer et envoyer les modifications avec Git (README, section 9)